

SDC 학생강사

컴퓨터공학 입문자를 위한 자료구조론

4주차

CONTENTS

01

지난 시간 문제 분석

해당 목차에 대한 설명을 입력해 주세요

02

트리

해당 목차에 대한 설명을 입력해 주세요

01

지난 시간 문제 분석

문제 1

에디터 다국어

☆ 한국어 ▾

시간 제한	메모리 제한	제출	정답	맞힌 사람	정답 비율
0.3 초 (하단 참고)	512 MB	153409	45025	30250	27.879%

문제

한 줄로 된 간단한 에디터를 구현하려고 한다. 이 편집기는 영어 소문자만을 기록할 수 있는 편집기로, 최대 600,000글자까지 입력할 수 있다.

이 편집기에는 '커서'라는 것이 있는데, 커서는 문장의 맨 앞(첫 번째 문자의 왼쪽), 문장의 맨 뒤(마지막 문자의 오른쪽), 또는 문장 중간 임의의 곳(모든 연속된 두 문자 사이)에 위치할 수 있다. 즉 길이가 L인 문자열이 현재 편집기에 입력되어 있으면, 커서가 위치할 수 있는 곳은 L+1가지 경우가 있다.

이 편집기가 지원하는 명령어는 다음과 같다.

L	커서를 왼쪽으로 한 칸 옮김 (커서가 문장의 맨 앞이면 무시됨)
D	커서를 오른쪽으로 한 칸 옮김 (커서가 문장의 맨 뒤이면 무시됨)
B	커서 왼쪽에 있는 문자를 삭제함 (커서가 문장의 맨 앞이면 무시됨) 삭제로 인해 커서는 한 칸 왼쪽으로 이동한 것처럼 나타나지만, 실제로 커서의 오른쪽에 있던 문자는 그대로임
P \$	\$라는 문자를 커서 왼쪽에 추가함

초기에 편집기에 입력되어 있는 문자열이 주어지고, 그 이후 입력한 명령어가 차례로 주어졌을 때, 모든 명령어를 수행하고 난 후 편집기에 입력되어 있는 문자열을 구하는 프로그램을 작성하시오. 단, 명령어가 수행되기 전에 커서는 문장의 맨 뒤에 위치하고 있다고 한다.

입력

첫째 줄에는 초기에 편집기에 입력되어 있는 문자열이 주어진다. 이 문자열은 길이가 N이고, 영어 소문자로만 이루어져 있으며, 길이는 100,000을 넘지 않는다. 둘째 줄에는 입력할 명령어의 개수를 나타내는 정수 $M(1 \leq M \leq 500,000)$ 이 주어진다. 셋째 줄부터 M개의 줄에 걸쳐 입력할 명령어가 순서대

트리

자료구조는 크게 선형자료구조, 비선형 자료구조로 나뉜다.

지난 시간까지 배운 자료는 선형구조(큐, 리스트, 스택)

비선형 자료구조로 가장 유명한 트리, 힙을 배울 예정



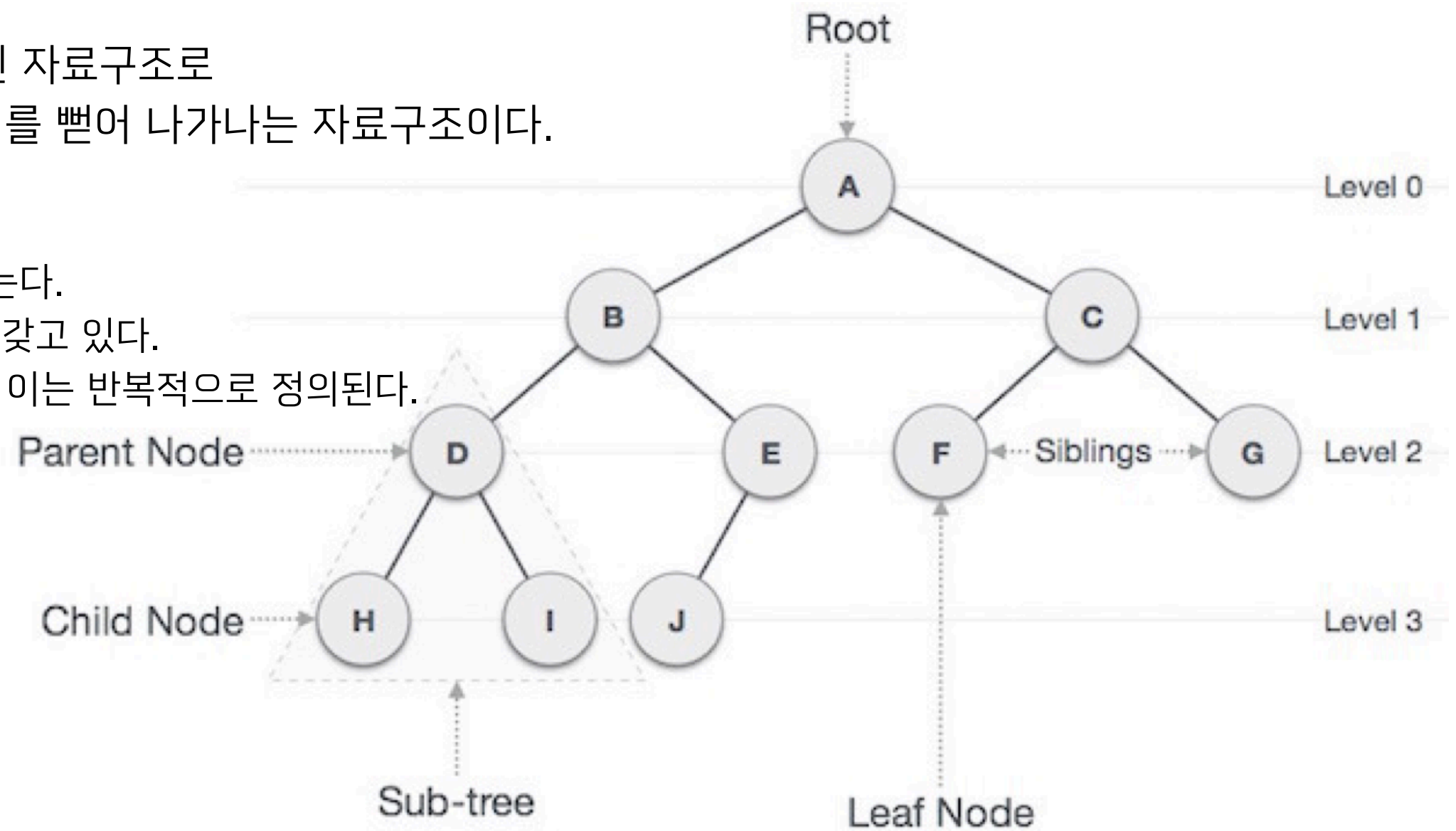
02

트리

트리

트리는 위계(Hierarchy) 를 가진 자료구조로
제일 상위노드(Root 노드)를 기준으로 아래로 가지를 뻗어 나가나는 자료구조이다.

1. 트리는 하나의 루트 노드를 갖는다.
2. 루트 노드는 0개 이상의 자식 노드를 갖고 있다.
3. 그 자식 노드 또한 0개 이상의 자식 노드를 갖고 있고, 이는 반복적으로 정의된다.

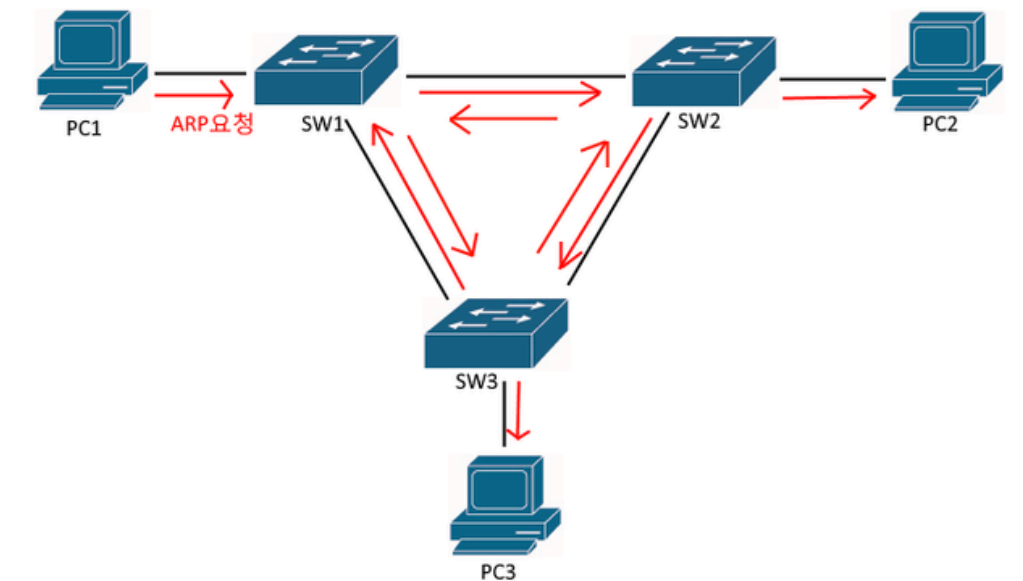
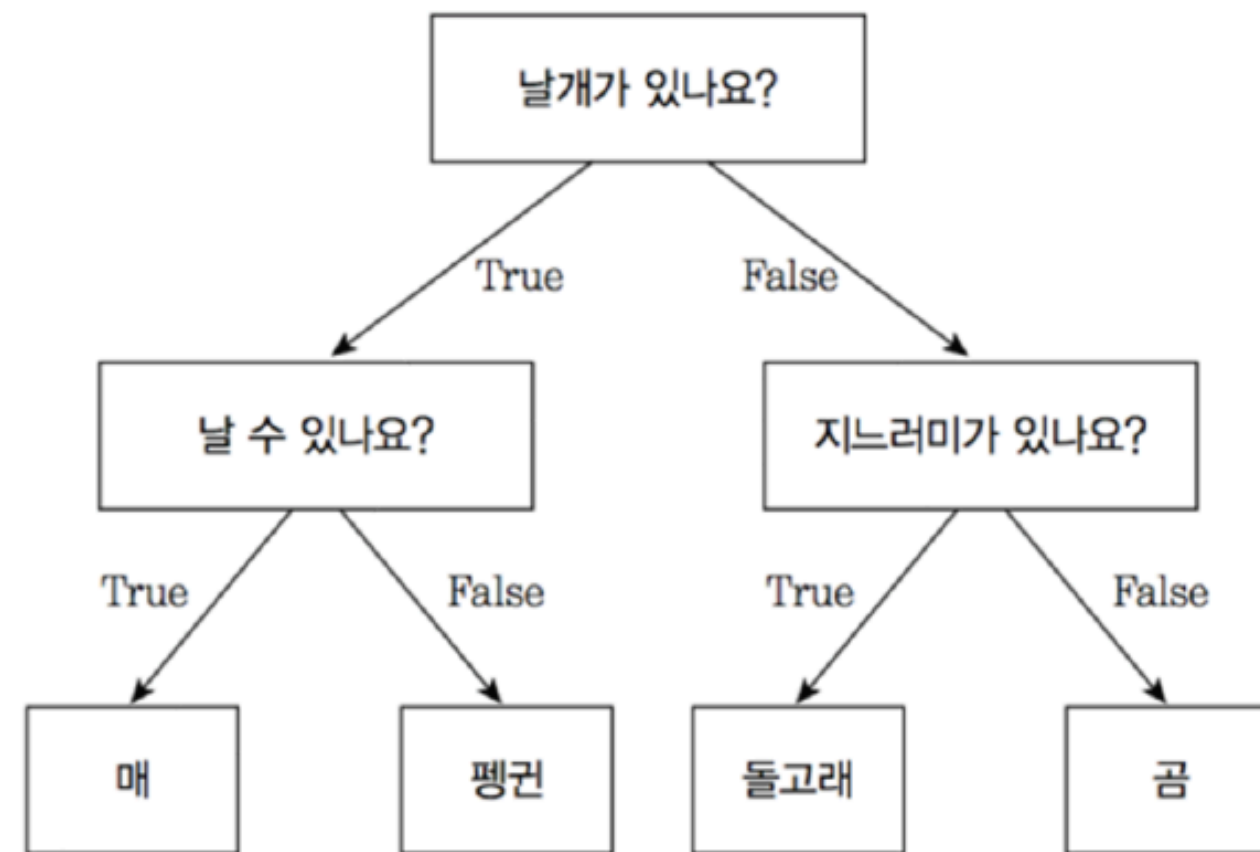
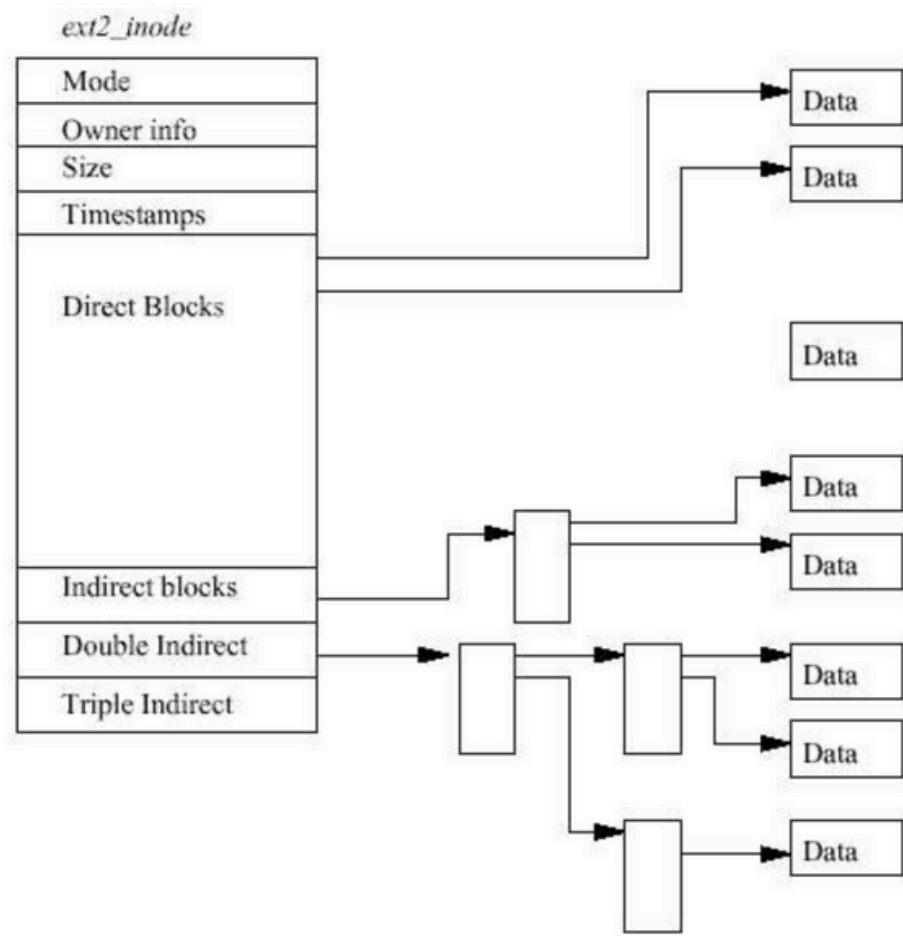


02

트리

트리

파일 시스템 관리, ML에서 의사결정 트리, 네트워크 STP, 데이터베이스 인덱싱등 매우 많은 CS에서 트리 구조를 활용함

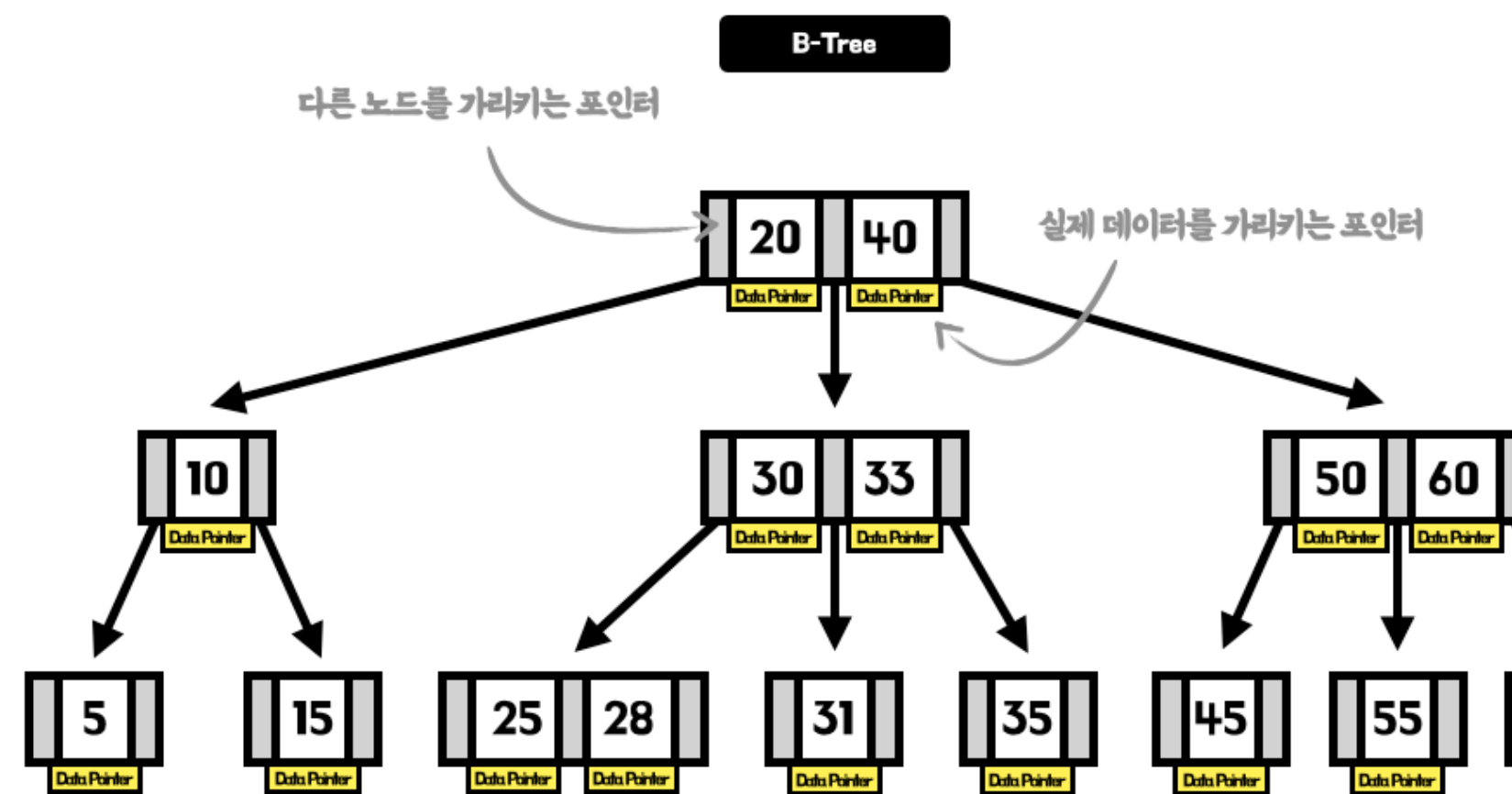
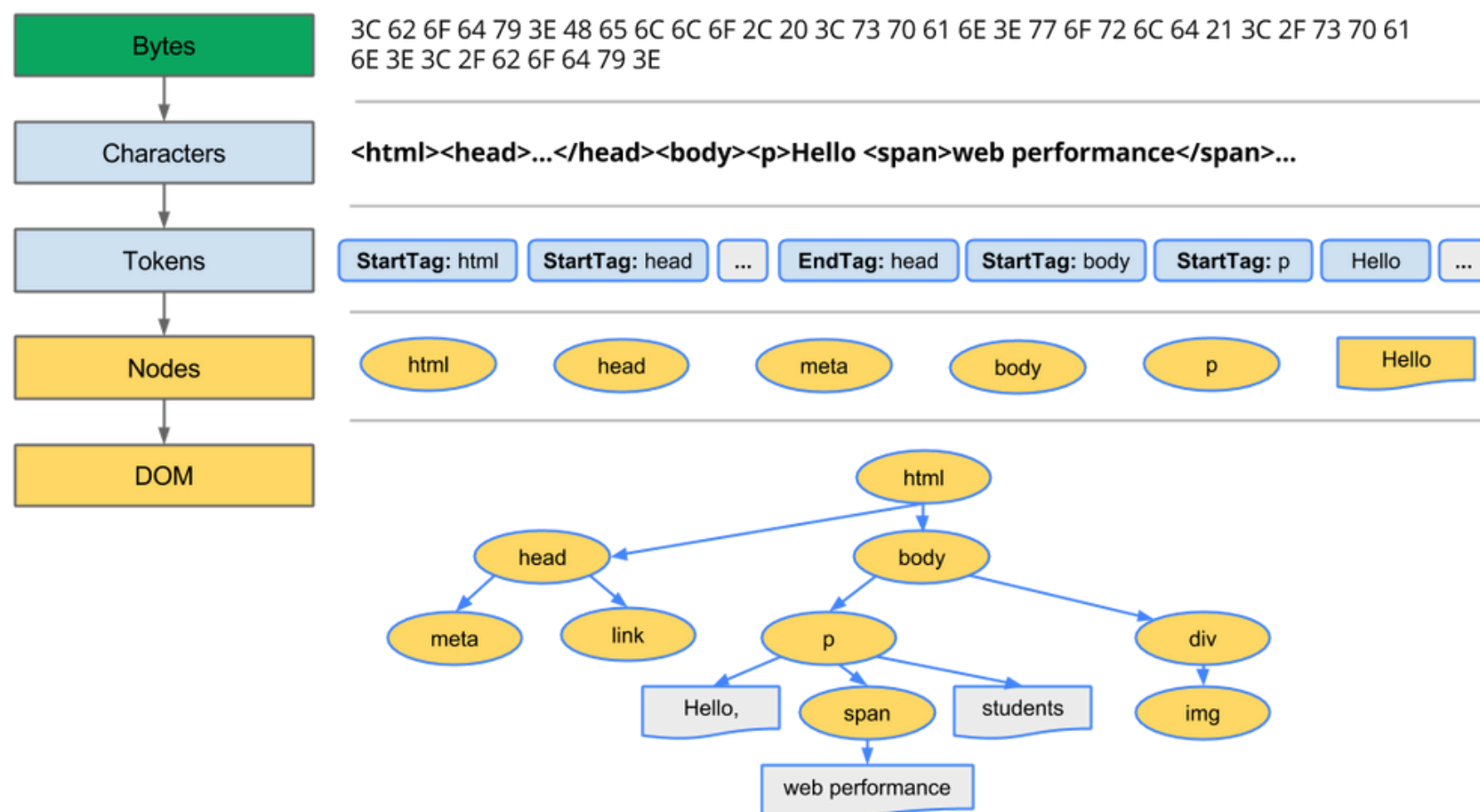


02

트리

트리

파일 시스템 관리, ML에서 의사결정 트리, 네트워크 STP, 데이터베이스 인덱싱등 매우 많은 CS에서 트리 구조를 활용함

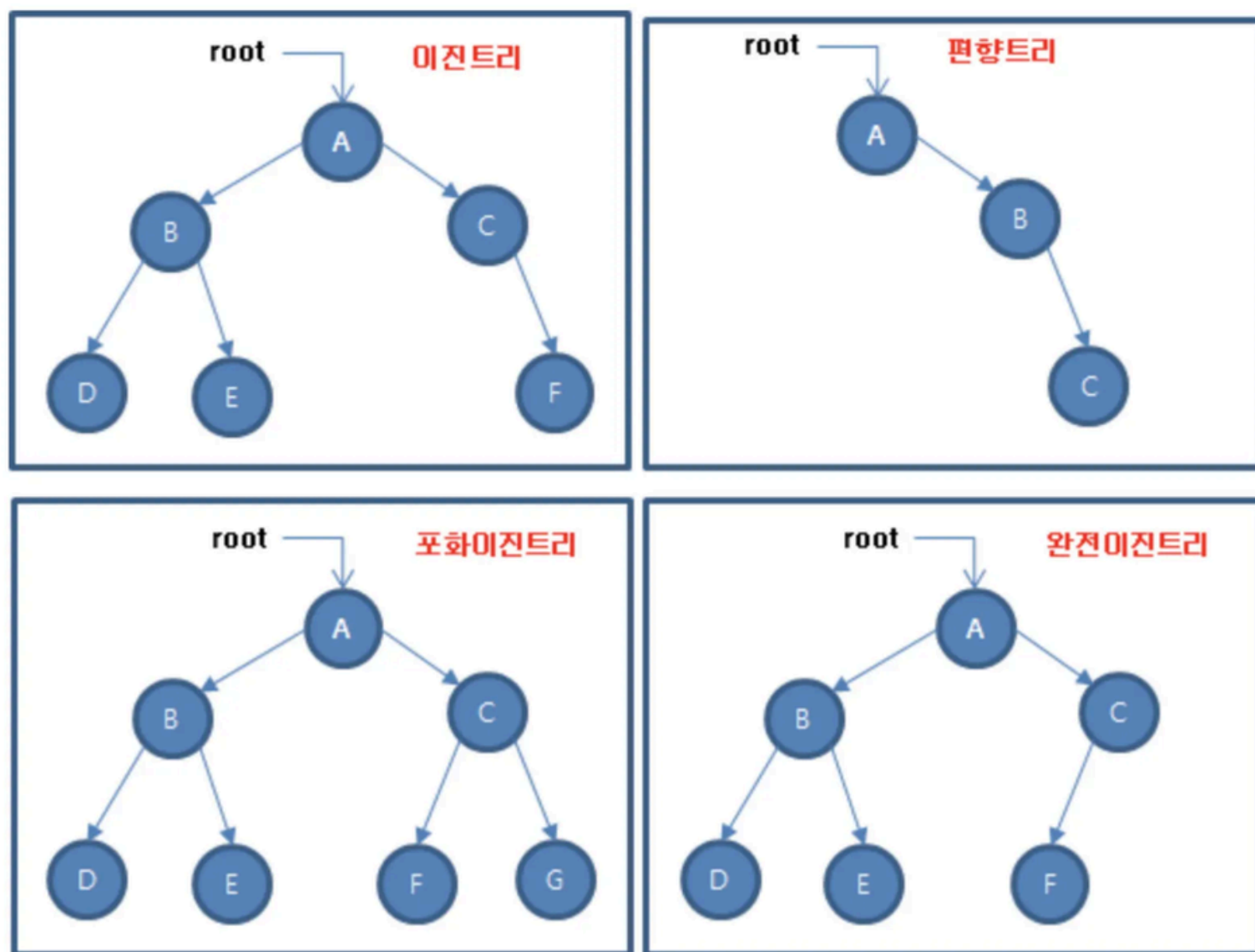


02

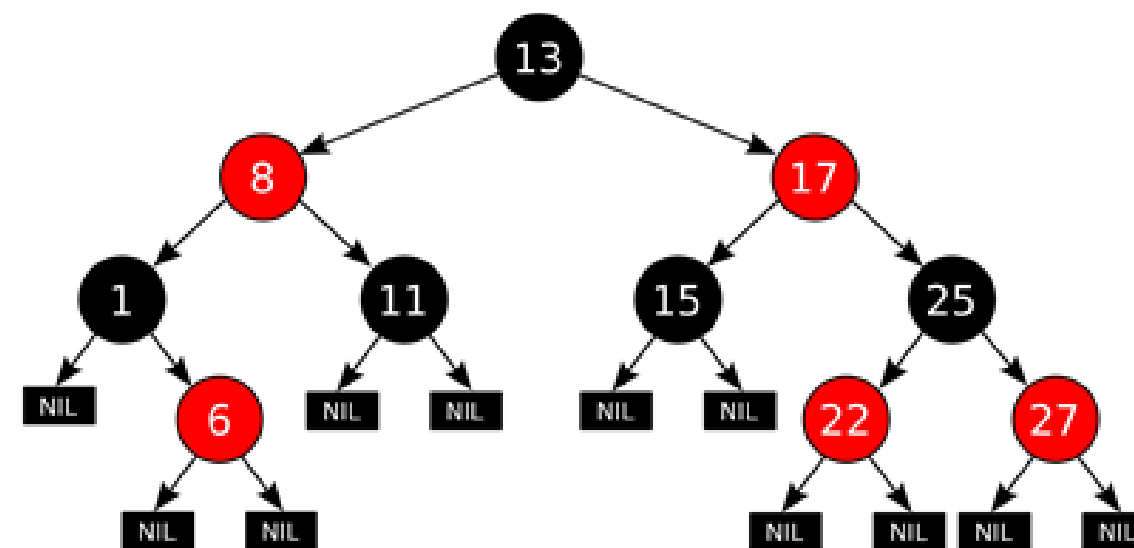
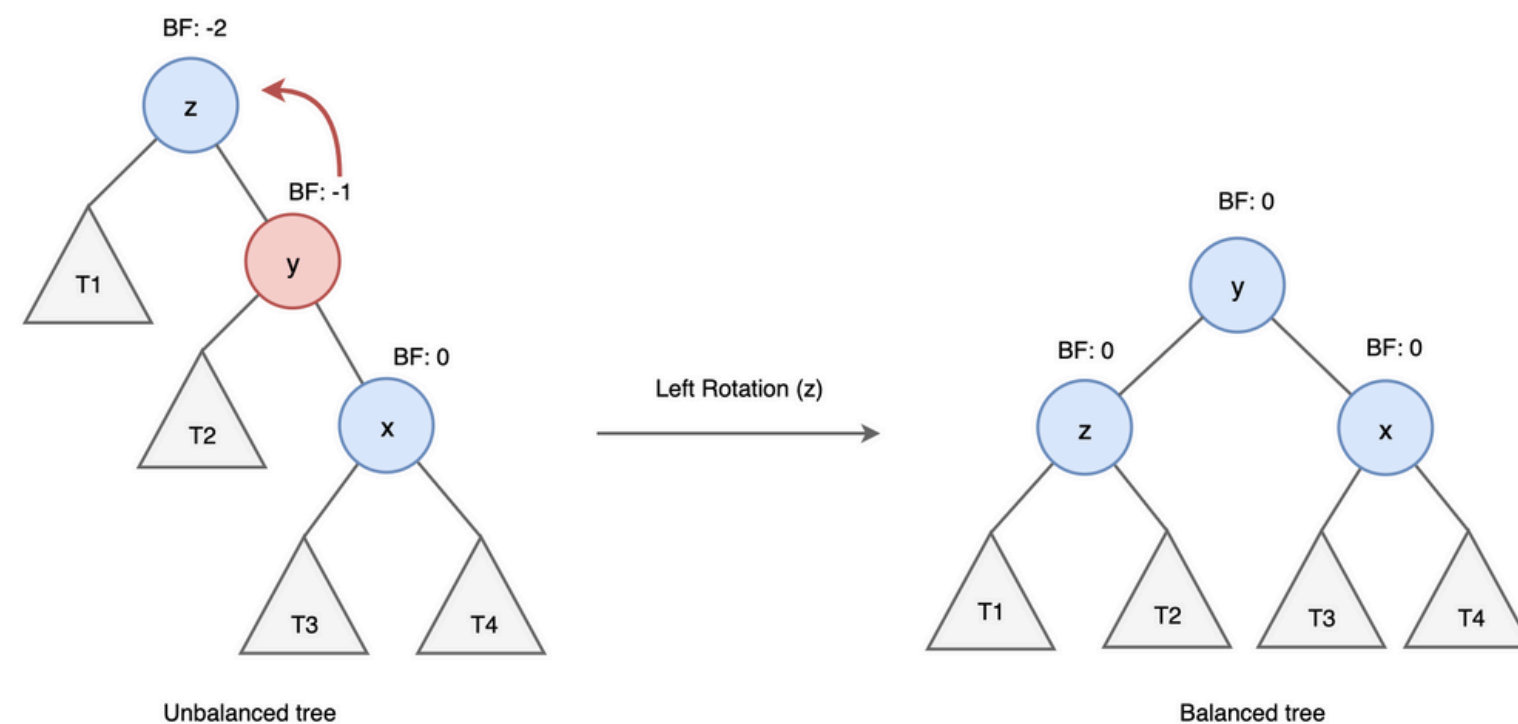
트리

트리

트리 자체도 여러가지 종류가 존재함



이진 트리 종류

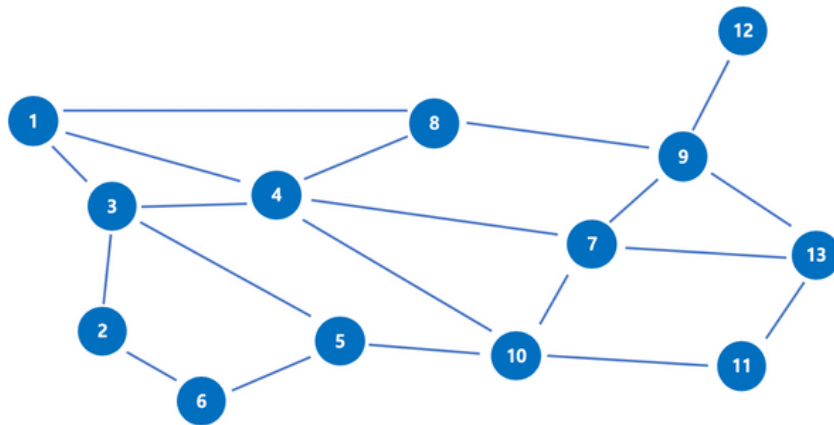


02

트리

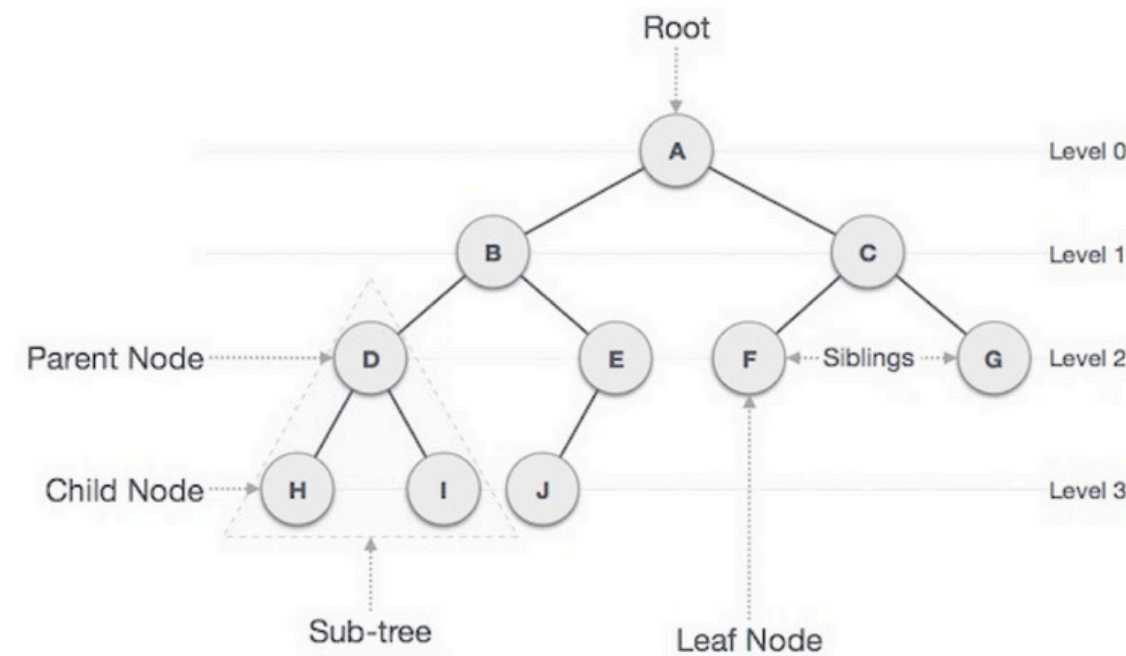
트리

트리는 특수한 형태의 그래프이다.
그래프 $\supset$ 트리 $\supset$ 힙



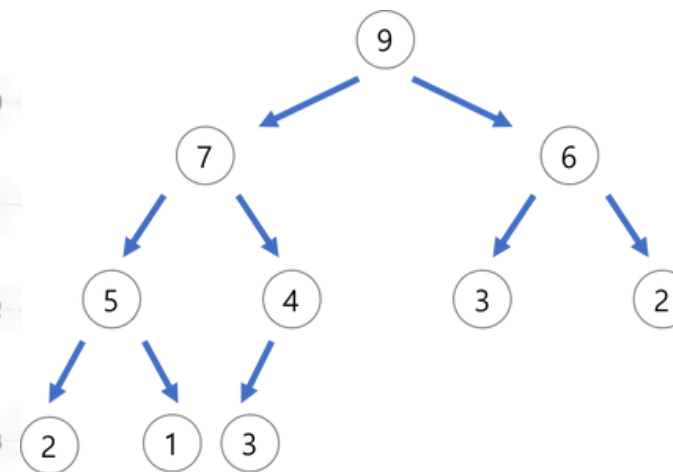
그래프(Graph)

- 노드(정점)와 엣지(간선)로 구성된 가장 일반적인 구조
- 사이클이 있어도 되고, 방향이 있어도/없어도 됨
- 연결이 자유로움

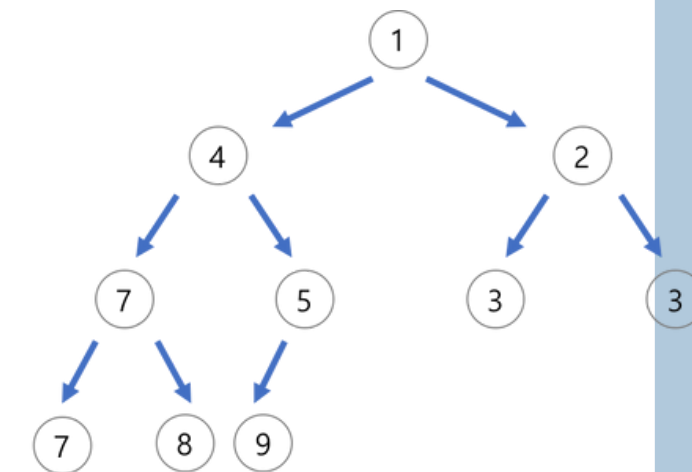


트리(Tree)

- 사이클이 없고(acyclic), 연결되어 있으며(connected),
- n 개의 노드에 $n-1$ 개의 엣지를 가짐
- 루트 노드가 있고 계층 구조를 가짐



-최대 힙(max heap)-



-최소 힙(min heap)-

힙(Heap)

- 완전 이진 트리(Complete Binary Tree) 구조여야 함
- 힙 속성을 만족해야 함
- (최대 힙: 부모 $\geq$ 자식, 최소 힙: 부모 $\leq$ 자식)

02

트리

트리의 용어

트리는 노드(Node, vertex, element)와 각 Node를 연결하는 Edge(간선)으로 구성되어 있다.

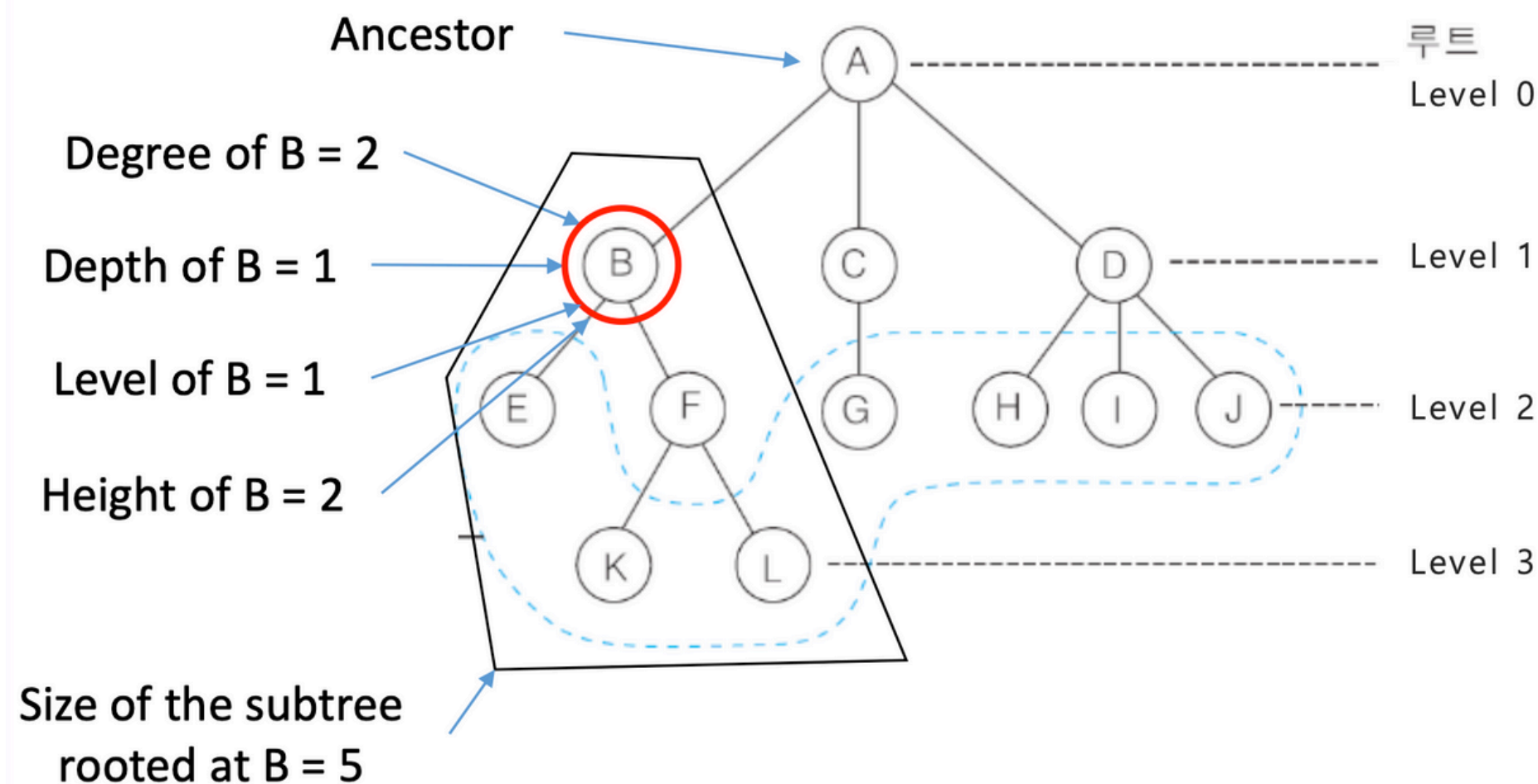
- Root(루트 노드) : 최상위 노드. 즉, 부모가 없는 노드. 일반적으로 트리의 access point 는 Root이다.
- Internal node(내부 노드): child가 적어도 하나 있는 노드
- External node(단말 노드): 위치상 맨 끝 노드를 의미하며 leaf 라고도 불린다.
- Ancestors of a node(조상 노드) : 한 노드의 모든 상위 노드. 직속 상위인 parent 뿐만 아니라 root 까지의 모든 상위노드. ex. parent, grandparent, great-grandparent 등
- Descendant of a node(자손 노드) : 한 노드의 모든 하위노드. 직속 하위뿐 아니라 말단 노드까지의 모든 하위노드
- Depth (node) (깊이) : 한 노드의 조상 수
- Height (tree) (높이): 트리의 노드 중 Depth의 최댓값
- Subtree (=부분트리) : 한 노드와 그 노드의 후손을 전부 포함하는 트리의 요소

02

트리

트리의 요소

- 노드의 크기(size): 자신을 포함한 모든 자손 노드의 개수
노드의 차수(degree): 해당 노드가 가지는 자식의 수
노드의 레벨(level): 트리의 특정 깊이를 가지는 노드의 집합
노드의 깊이(depth): 루트에서 자신까지 가는 경로의 길이
root 노드의 depth는 0
노드의 높이(height): 자신과 자손 중 단말 노드 사이의 경로의 최대 길이
leaf 노드의 height는 0



02

트리

트리 ADT

size - 트리 전체의 Node 개수

empty - 비어있는 트리인지 아닌지

root - 루트 노드를 반환

positions - 각 노드의 주소를 저장

parent - 노드의 부모 노드를 반환

children - 노드의 자식을 배열로 반환

Tree ADT

- We use positions to abstract nodes
 - Generic methods:
 - integer **size()**
 - boolean **empty()**
 - Accessor methods:
 - position **root()**
 - list<position> **positions()**
 - Position-based methods:
 - position **parent()**
 - list<position> **children()**
- ◆ Query methods:
 - boolean **isRoot()**
 - boolean **isExternal()**
 - ◆ Additional update methods may be defined by data structures implementing the Tree ADT

02

트리

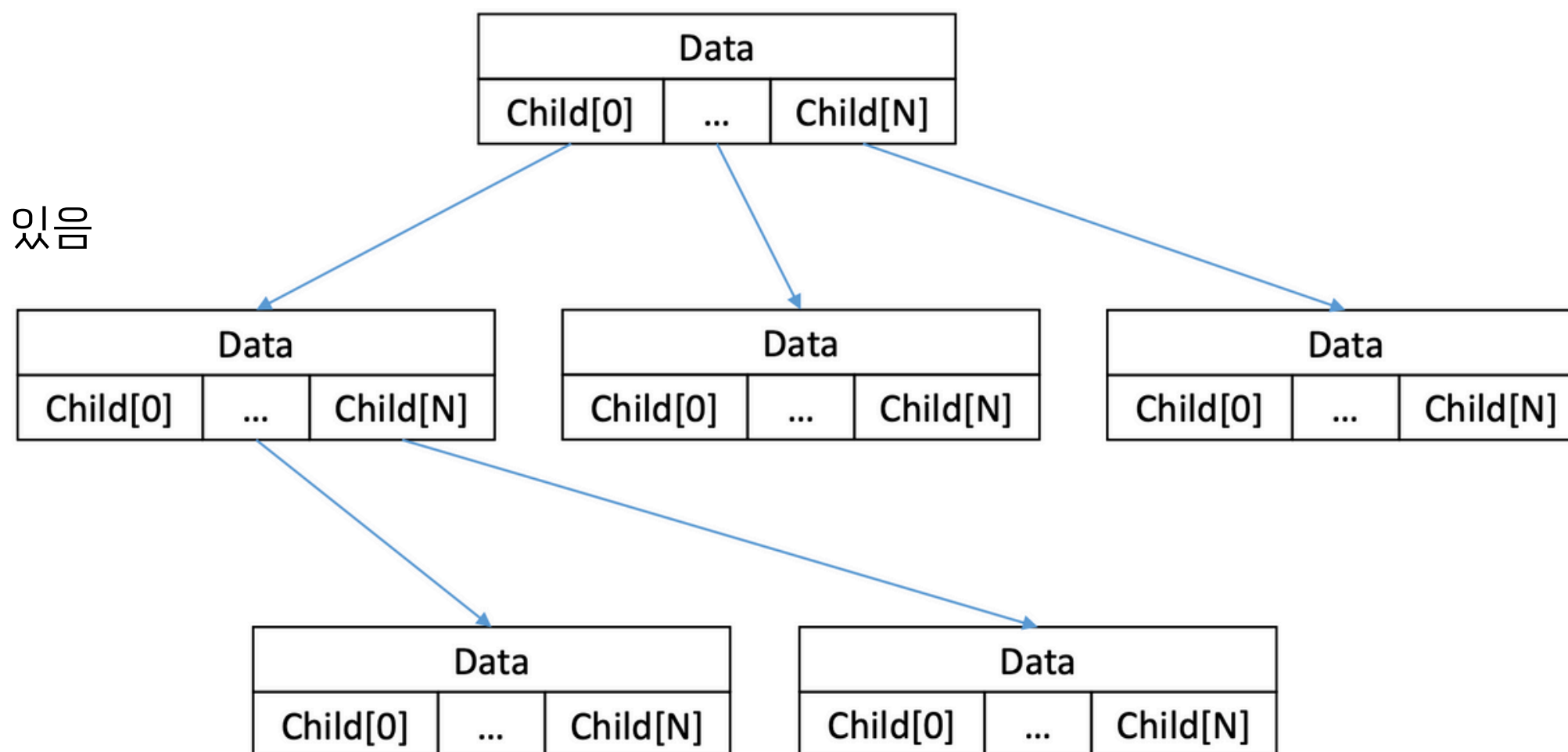
트리의 표현 방식

각 노드는 child 주소를 배열로 가지고 있음

자식 주소로 Node를 포인팅 하는 방법을 사용

자식을 무한히 가져야하는 경우 구현이 복잡 할 수 있음

➤ N 링크 방법



02

트리

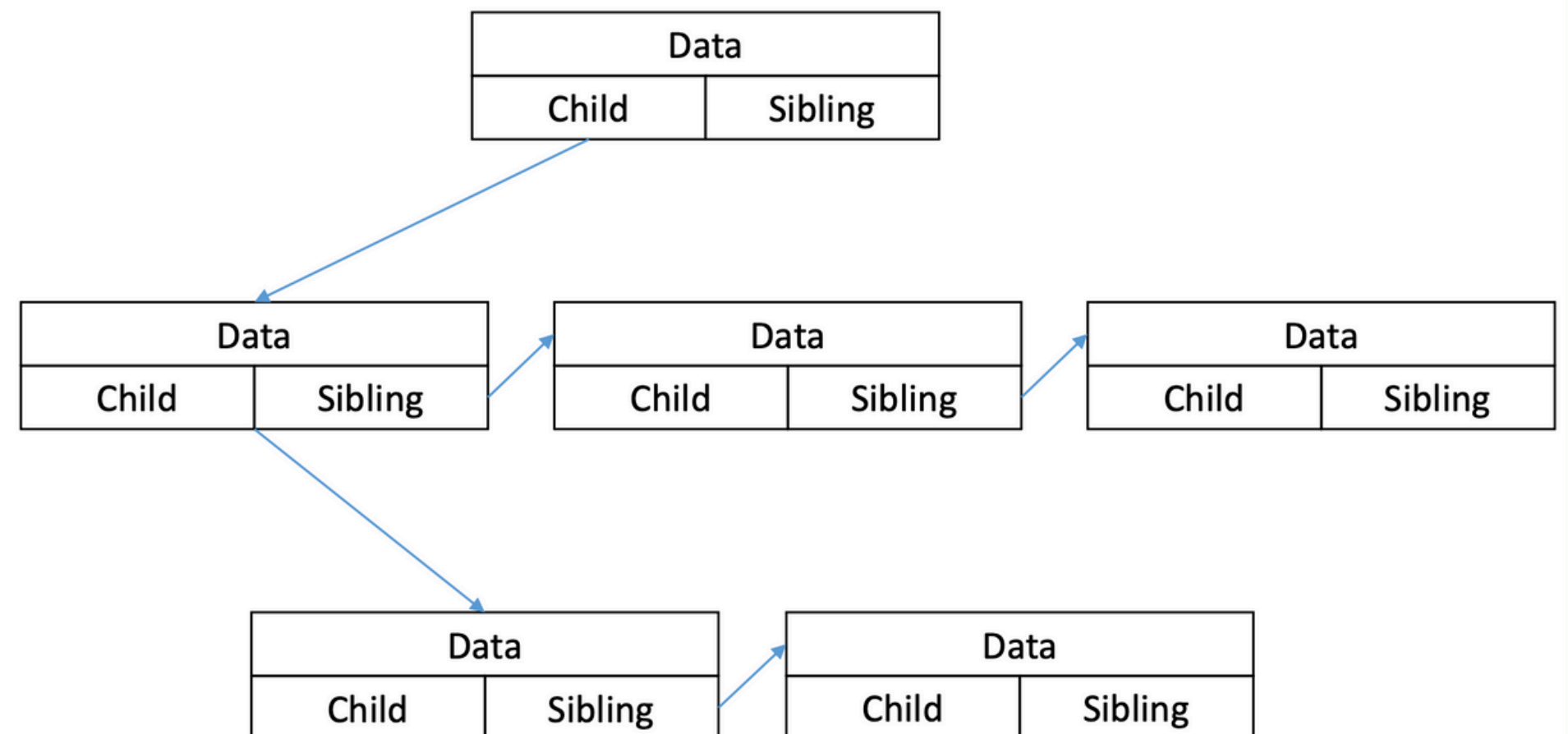
트리의 표현 방식

각 노드는 Child, Sibling 포인터만 가지고 있음

직관적인 방법은 아니지만 구현이 편리함

다만 삽입, 조회, 삭제 연산이 더 오래걸림

▶ Left Child – Right Sibling 방법



02

트리

STL <vector>

N링크로 구현하기 위해 STL vector를 사용

자동으로 배열의 크기 조절, 객체의 추가와 삭제가 가능한 동적 배열 구조

push_back() - 새로운 원소를 맨 뒤에 삽입

[] - 배열과 똑같이 원소 접근

size() - 배열 내 원소의 개수를 반환

```
#include <vector>
```

```
using namespace std;
```

```
int main() {
```

```
vector<int> v; // < > 안에 벡터의 타입을 명시해야 한다.
```

```
return 0;
```

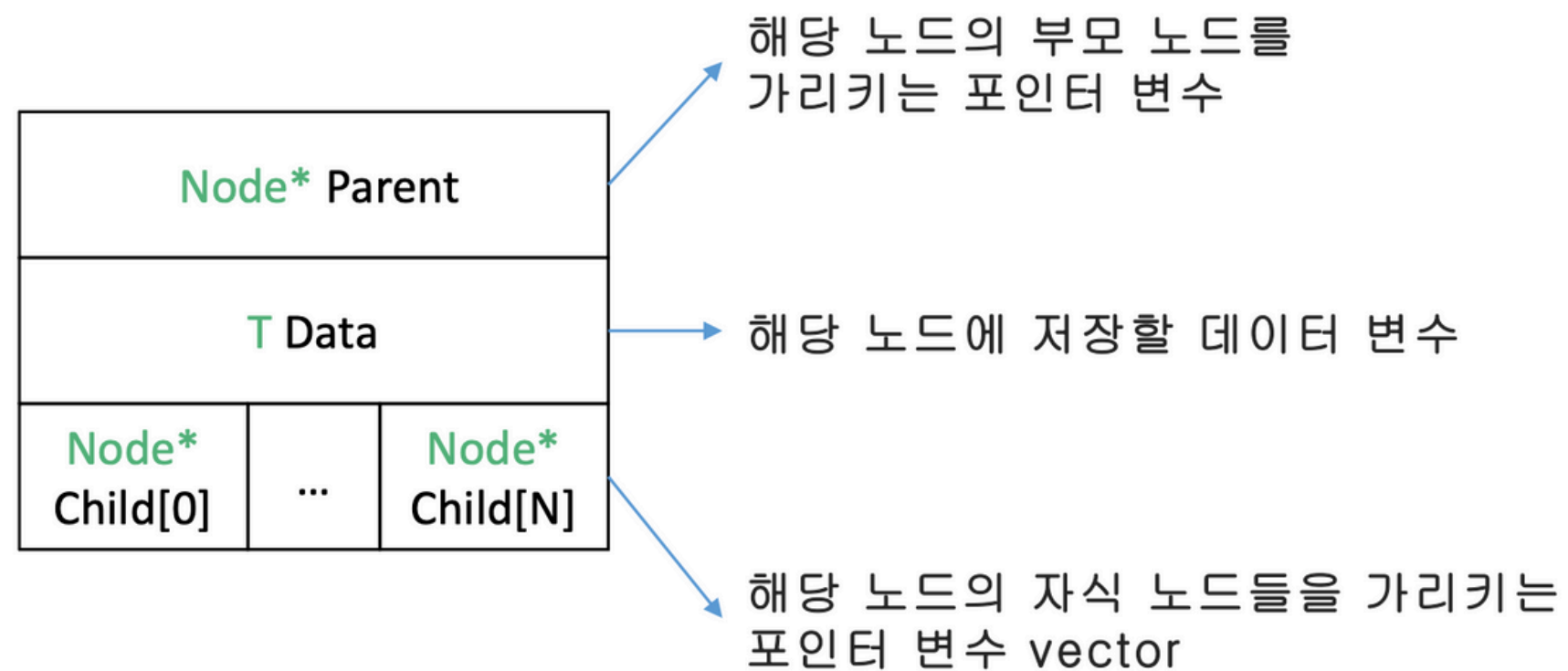
```
};
```

02

트리

트리 구현

트리의 기본이 되는 Node 클래스를 우선 구현



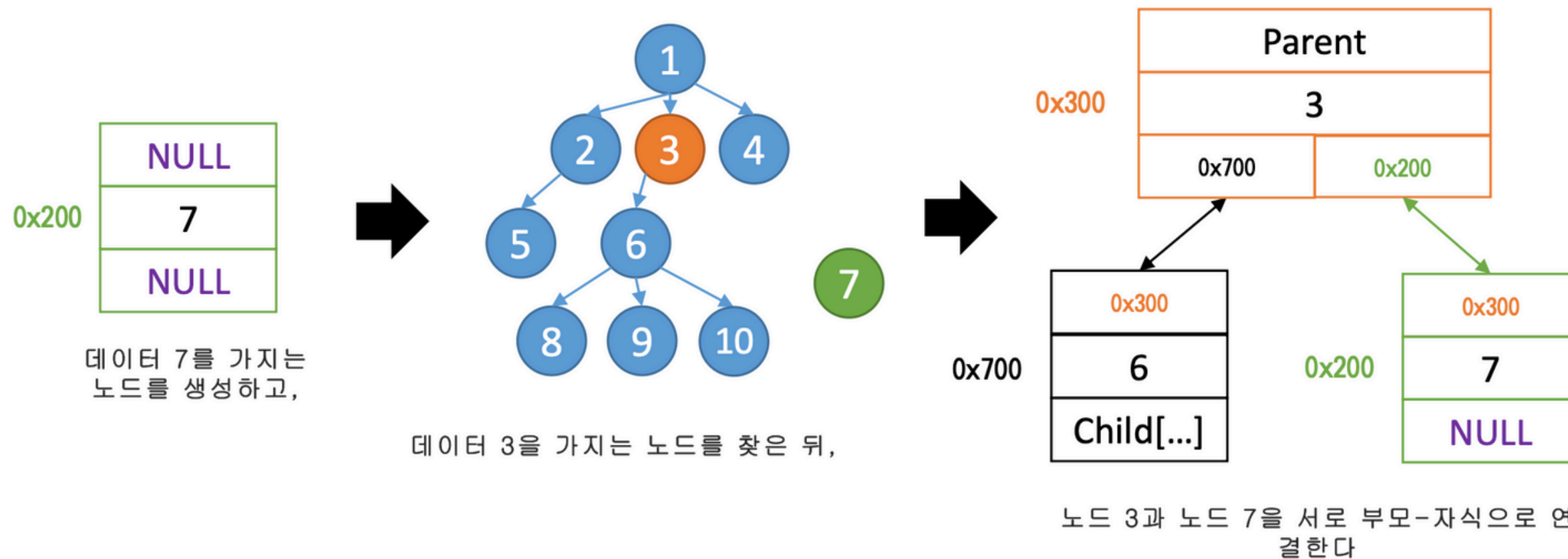
02

트리

트리 구현

삽입 연산

➤ 예시) 노드 3의 자식으로 노드 7을 삽입



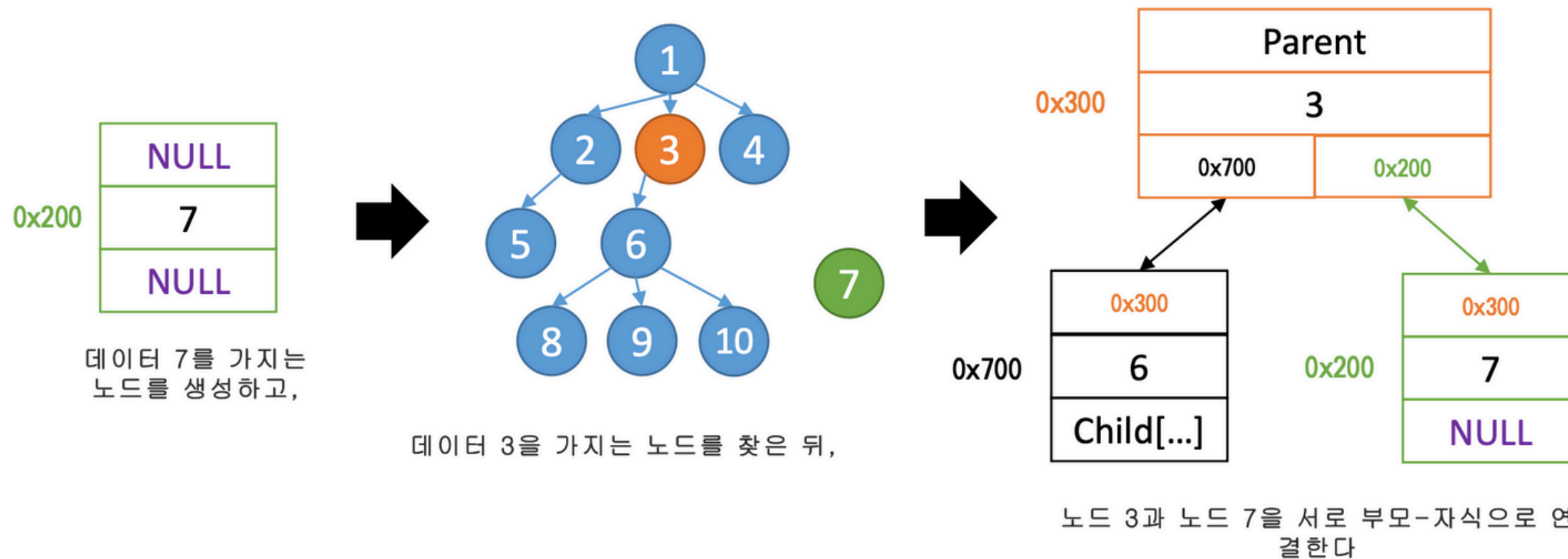
02

트리

트리 구현

삽입 연산

➤ 예시) 노드 3의 자식으로 노드 7을 삽입



02

트리

트리 구현

```
1 #include <vector>
2 using namespace std;
3
4 class Node
5 {
6 public:
7     int data;
8     Node *parent;
9     vector<Node *> child;
10
11     Node();
12     Node(int data);
13     ~Node();
14
15     void insertChild(Node *child);
16     void delChild(Node *child);
17 };
18
```

```
1
2 #include <vector>
3 #include <node.h>
4 using namespace std;
5
6 class Tree
7 {
8 private:
9     Node *root;
10    vector<Node *> node_list;
11
12    void deleteSubTree(Node *node);
13
14 public:
15     Tree();
16     Tree(int data);
17     ~Tree();
18
19     void setRoot(int data);
20     Node *getRoot();
21
22     void insertNode(int parent_data, int data);
23     void delNode(int data);
24
25     Node *findNode(int data);
26     void printChild(int data);
27     int countdepth(int data);
28 };
29
```

02

트리

이진 트리

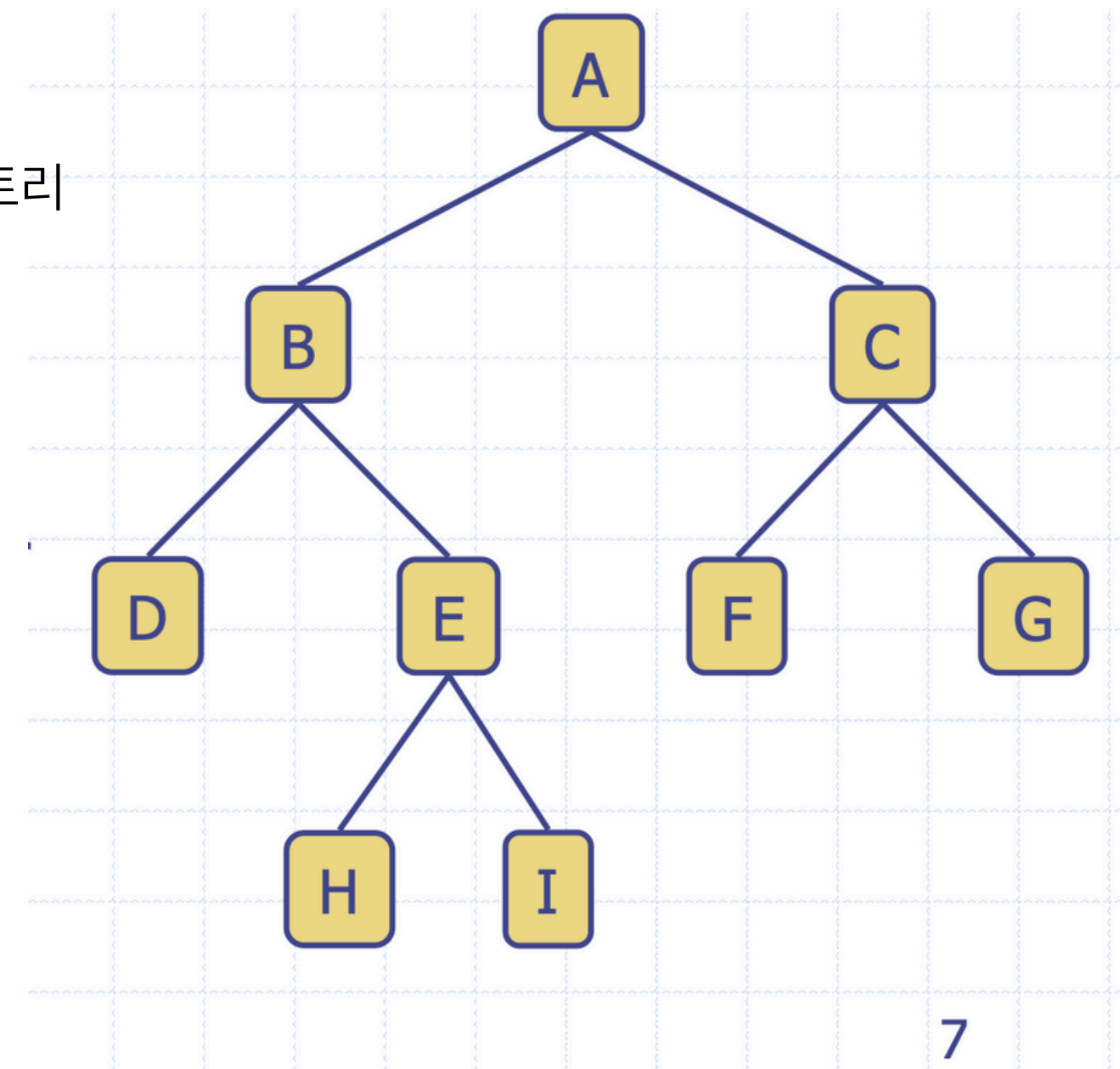
이진트리는 트리의 일종으로 자식을 최대 2개만 가지는 트리임

1. 정 이진 트리 (Full Binary Tree) - 모든 노드가 0개 또는 2개의 자식을 가지는 트리

2. 완전 이진 트리 (Complete Binary Tree)
마지막 레벨을 제외한 모든 레벨이 완전히 채워져 있고,
마지막 레벨은 왼쪽부터 순서대로 채워진 트리입니다.

3. 포화 이진 트리 (Perfect Binary Tree)
모든 레벨이 완전히 채워진 트리입니다.

4. 균형 이진 트리 (Balanced Binary Tree)
모든 노드에서 왼쪽 서브트리와 오른쪽 서브트리의 높이
차이가 1 이하인 트리입니다.



이진 트리

BinaryTree ADT

- The BinaryTree ADT extends the Tree ADT, i.e., it inherits all the methods of the Tree ADT
- Additional methods:
 - position p.left()
 - position p.right()
- Update methods may be defined by data structures implementing the BinaryTree ADT
- **Proper binary tree:** Each node has either 0 or 2 children

02

트리

이진 트리의 성질

N - 노드의 전체 개수

E - 외부(리프) 노드의 개수

I - 내부 노드의 개수

H - 트리의 높이

$$\blacksquare e = i + 1$$

$$\blacksquare n = 2e - 1$$

$$\blacksquare h \leq i$$

$$\blacksquare h \leq (n - 1)/2$$

$$\blacksquare e \leq 2^h$$

$$\blacksquare h \geq \log_2 e$$

$$\blacksquare h \geq \log_2 (n + 1) - 1$$

높이는 내부 노드의 수를 넘을수 없다

이진트리에서는 각 H별로
최대 노드의 개수가 정해져있음

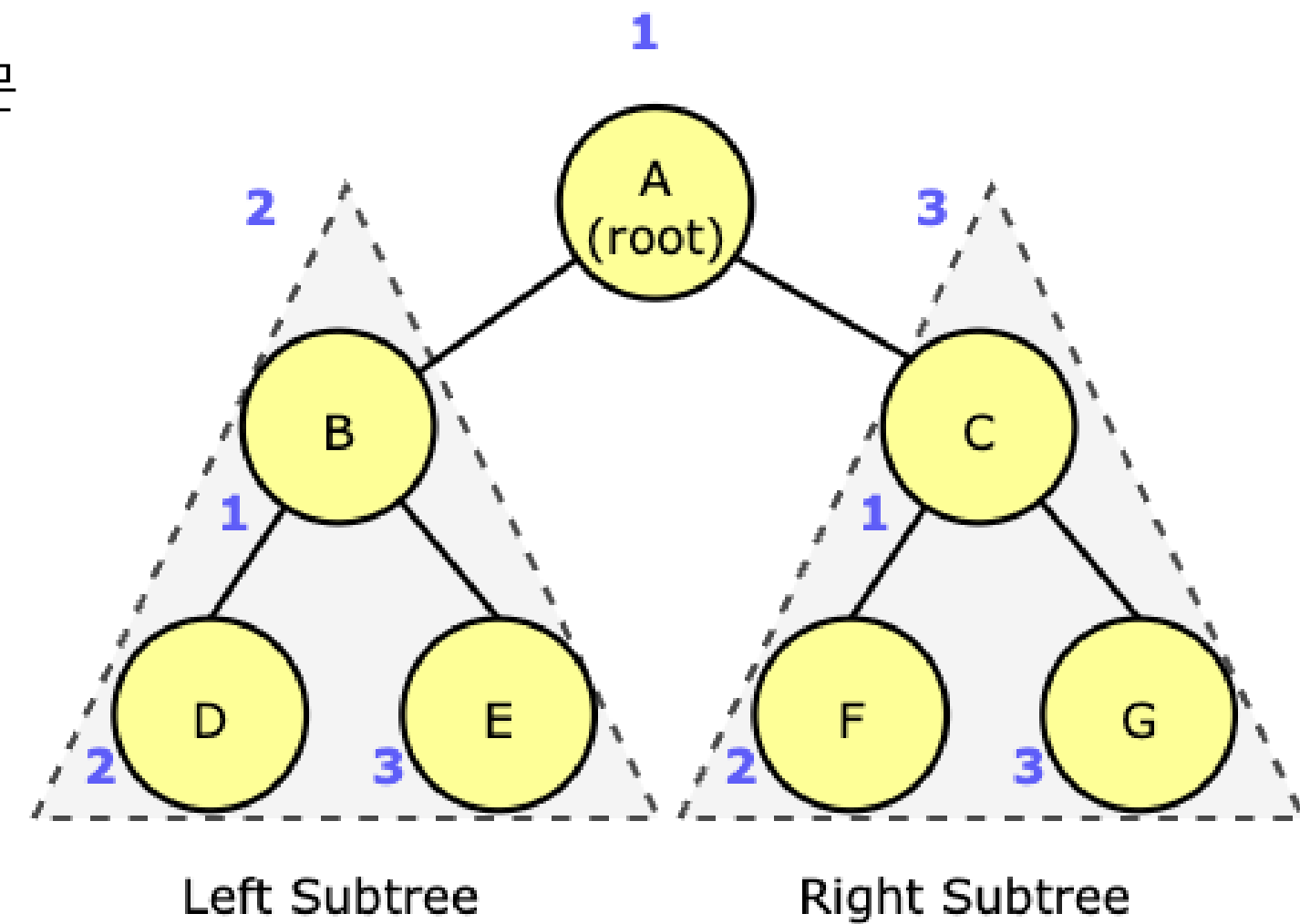
02

트리

트리 순회

전위 순회(Preorder Traversal)

전위 순회는 루트 노드를 가장 먼저 방문하고, 그 다음에 왼쪽 자식을 방문한 후 오른쪽 자식을 방문하는 방식



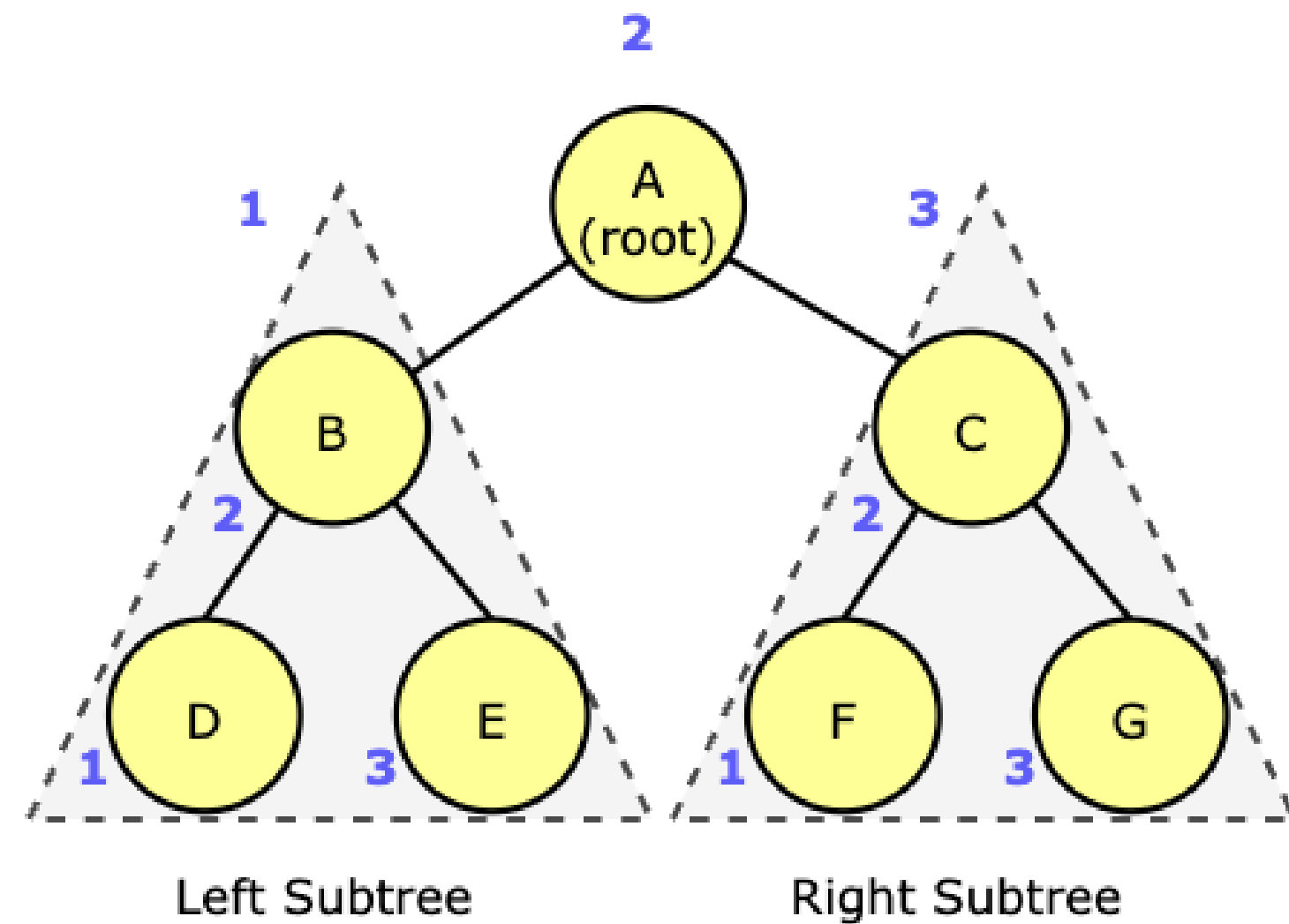
02

트리

트리 순회

중위 순회(Inorder Traversal)

중위 순회는 왼쪽 자식을 먼저 방문한 후 루트 노드를 방문하고,
그 다음 오른쪽 자식을 방문하는 방식



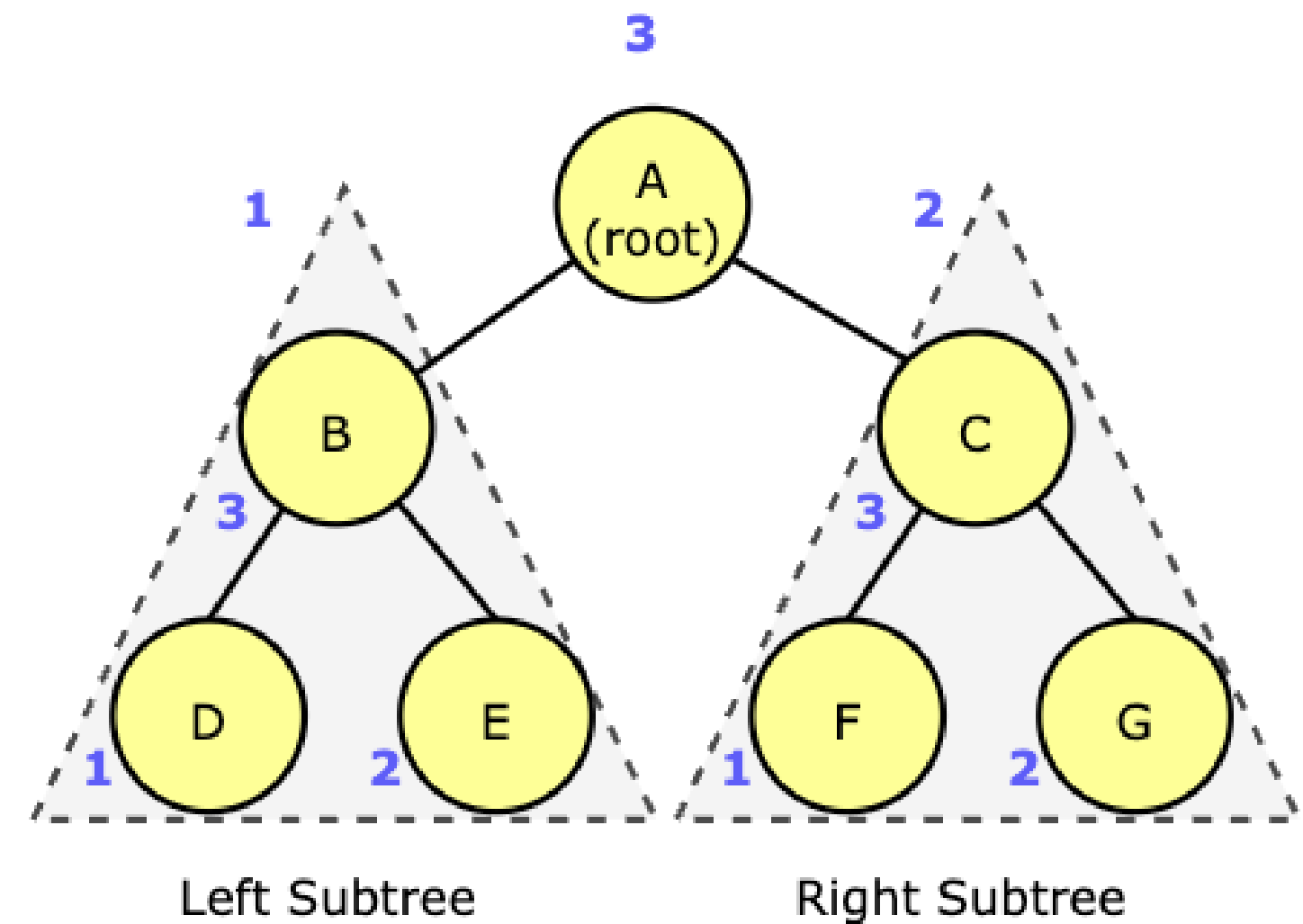
02

트리

트리 순회

후위 순회(Postorder Traversal)

후위 순회는 왼쪽 자식과 오른쪽 자식을 먼저 모두 방문한 후에
루트 노드를 마지막에 방문하는 방식



02

트리

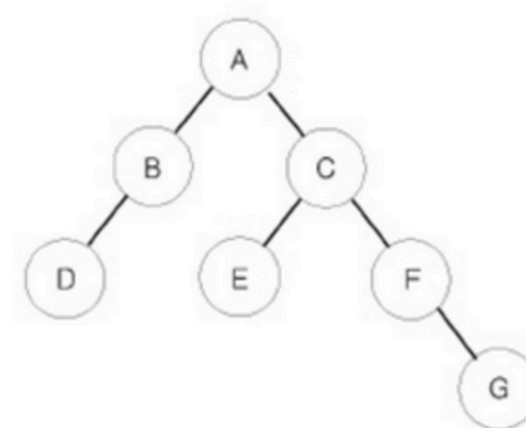
트리 순회

트리 순회

시간 제한	메모리 제한	제출	정답	맞힌 사람	정답 비율
2 초	128 MB	77478	50903	39241	67.473%

문제

이진 트리를 입력받아 전위 순회(preorder traversal), 중위 순회(inorder traversal), 후위 순회(postorder traversal)한 결과를 출력하는 프로그램을 작성하시오.



예를 들어 위와 같은 이진 트리가 입력되면,

- 전위 순회한 결과 : ABDCEFG // (루트) (왼쪽 자식) (오른쪽 자식)
- 중위 순회한 결과 : DBAECFG // (왼쪽 자식) (루트) (오른쪽 자식)
- 후위 순회한 결과 : DBEGFCA // (왼쪽 자식) (오른쪽 자식) (루트)

<https://www.acmicpc.net/problem/1158>

백준 1158번 요세푸스 순열을 풀어오기
단, 오늘 배운 Sequence 자료구조를 활용해서 풀어오기
구현방법은 자유

STL 사용금지
못풀어도 이렇게 생각을 해봤다 정도면 좋음